

Assignment 2: AWS Compute Services

ComS/CprE 4590X/5590 2026 Spring

Total: 30 points

Due: **Friday, February 20, 11:59 PM**

Lab 1: Auto-Scaled HTTPS Service Using Self-Signed Certificates (15 pts)

Learning Objectives

Upon completion of this lab, students will be able to:

- Deploy a network-accessible service on Amazon EC2
- Automate instance configuration using launch-time user data scripts
- Create and use Launch Templates and Auto Scaling Groups (ASGs)
- Configure a load-balanced HTTPS service using self-signed certificates
- Understand TLS trust, certificate warnings, and encryption in practice
- Validate scaling and fault tolerance of cloud-based services

Overview

In this lab, you will deploy a simple HTTPS service on multiple EC2 instances and place them behind an Elastic Load Balancer (ELB). The service uses a **self-signed TLS certificate**, allowing HTTPS communication without requiring a registered domain name or AWS Certificate Manager (ACM). Self-signed certificates are commonly used in development and testing environments. Although browsers and clients do not trust them by default, they are sufficient for understanding how HTTPS works in practice.

Architecture

- Client → Load Balancer: HTTPS (TCP port 8443)
- Load Balancer → EC2 Instances: HTTPS (TCP port 8443)
- TLS termination occurs on the EC2 instances
- All instances use the same self-signed certificate

Part 1: Generate a Self-Signed Certificate (Local Machine)

On your local machine, generate a private key and self-signed certificate using `openssl`:

```
openssl req -x509 -newkey rsa:2048 -nodes -keyout server.key -out server.crt -days 365
```

When prompted, you may use any value for the **Common Name (CN)** (e.g., `localhost`) and leave other fields blank.

After this step, you should have two files:

- `server.key`: the private key
- `server.crt`: the self-signed certificate

Part 2: Understand and Test the HTTPS Service Locally

You are provided with a Python script `https_server.py` that:

- Listens on TCP port 8443
- Uses `server.key` and `server.crt`
- Responds with hostname, private IP, and timestamp

Place `https_server.py`, `server.key`, and `server.crt` in the same directory and run:

```
python3 https_server.py
```

Test the service locally using:

```
curl -k https://localhost:8443
```

The `-k` option tells `curl` to ignore certificate trust warnings.

Part 3: Launch and Validate a Single EC2 Instance

1. Launch an EC2 instance in the default VPC using Amazon Linux (free-tier eligible).
2. Configure a security group that allows:
 - SSH (TCP 22) from your IP address
 - HTTPS (TCP 8443) from anywhere
3. Upload `https_server.py`, `server.key`, and `server.crt` to the instance.
4. Start the server:

```
python3 https_server.py
```

5. From your local machine, verify connectivity:

```
curl -k https://<EC2-public-IP>:8443
```

Part 4: Create a Launch Template

Create a Launch Template that:

- Uses Amazon Linux
- Attaches the security group from Part 3
- Includes a **user data script** (provided; make sure you understand how it works) that:
 - Installs Python 3
 - Writes the certificate, key, and server script to disk
 - Starts the HTTPS server automatically at boot

Launch a test instance using the template and verify HTTPS access without manual login.

Part 5: Create an Auto Scaling Group

Create an Auto Scaling Group using the Launch Template with the following configuration:

- Minimum instances: 2
- Desired instances: 2
- Maximum instances: 3
- Use all subnets in the default VPC

Verify that multiple instances are launched automatically.

Part 6: Create an HTTPS Load Balancer

1. Create a Network Load Balancer (internet-facing).
2. Configure a listener on TCP port 8443.
3. Create a target group with:
 - Protocol: TCP
 - Port: 8443
 - Health check protocol: TCP

4. Attach the target group to the Auto Scaling Group.

Wait until all targets are marked **Healthy**.

Part 7: Client Testing via Load Balancer

Obtain the DNS name of the load balancer and test from your local machine:

```
curl -k https://<load-balancer-DNS>:8443
```

Repeat the request multiple times and confirm that responses originate from different instances.

Part 8: Failure and Recovery Test

Manually terminate one EC2 instance and observe that:

- The Auto Scaling Group launches a replacement
- The load balancer continues to serve HTTPS requests

Part 9: TA Validation and Teardown

Email the load balancer DNS name to the assigned TA (Md Anwar Hossen for 4590X students whose last names are A-N; Ming Liu for the rest) for validation. Do not tear down your resources until testing is complete. Once confirmed, delete all AWS resources immediately to avoid unnecessary charges.

Submission Requirements

Submit a single PDF containing clearly readable screenshots of:

- Launch Template details (Instance details and Advanced details)
- Security group inbound rules
- Auto Scaling Group details
- Target group **Health Checks** tab
- Load balancer **Listeners** tab
- Output of `curl -k https://<load-balancer-DNS>:8443`

Notes

- Certificate warnings are acceptable for this lab.
- Self-signed certificates should never be used in production systems.
- As HTTPS is run between user and EC2 instances, the load balancer and target group should use TCP (8443) instead of HTTPS as the protocol to communicate with front/backend parties.
- All AWS resources should be deleted after validation.

Lab 2: AWS Lambda and API Gateway

Learning Objectives

After completing this lab, students will be able to:

- Design and deploy AWS Lambda functions using Python.
- Parse JSON input and produce structured JSON output.
- Configure IAM roles and permissions for AWS services.
- Use AWS SDK (boto3) to query S3 metadata.
- Expose Lambda functions as RESTful APIs using Amazon API Gateway.

Total Points

15 points

Part 0: Warm-up (Practice – Not Graded)

Before working on the graded tasks, students are strongly encouraged to practice:

- Creating a Lambda function using Python.
- Enabling and invoking a Lambda Function URL.
- Passing JSON input via the Lambda *Test* tab.
- Invoking Lambda functions using `curl`.

These skills are required for successful completion of the graded parts.

Part 1: Lambda Function – Rectangle Computation (5 points)

Task Description

Create a Lambda function that computes the **area** and **perimeter** of a rectangle.

Input (JSON)

```
{
  "length": 5,
  "width": 3
}
```

Output (JSON)

```
{
  "area": 15,
  "perimeter": 16
}
```

Requirements

- Use **Python 3** as the Lambda runtime.
- Enable **Function URL** with authentication type set to **NONE**.
- Parse user input from `event["body"]`.
- Implement helper functions `calculate_area()` and `calculate_perimeter()`.
- Return results as a JSON-formatted response.

Local Invocation Example

```
curl <function_url>  
-H "Content-Type: application/json"  
-d '{"length":5, "width":3}'
```

Part 2: Lambda Function – S3 Bucket Count (5 points)

Task Description

Create a Lambda function that returns the **number of S3 buckets** owned by the user. Bucket names must **not** be exposed.

If the user has **zero** S3 buckets, the function must explicitly return `"bucket_count": "none"` to distinguish this case from execution errors.

IAM Role Setup

Go to IAM and select Roles. Create a new IAM role, and select AWS Service → Lambda. Attach to this role the following policies:

- AmazonS3ReadOnlyAccess
- AWSLambdaBasicExecutionRole

Attach this role to the Lambda function.

Input

This function takes **no user input**.

Output (JSON)

```
{ "bucket_count": 3 }
```

If no buckets exist:

```
{ "bucket_count": "none" }
```

Requirements

- Use `boto3.client("s3")` to list buckets.
- Count buckets without revealing bucket names.
- Handle the zero-bucket case explicitly.
- Enable **Function URL** with authentication type set to `NONE`.

Part 3: API Gateway Integration (5 points)

Task Description

Expose your Lambda function for Part 1 using **Amazon API Gateway**.

Steps

1. Create a **REST API** in API Gateway.
2. Create a resource (e.g., `/rectangle` or `/s3count`).
3. Add a **POST** method to the resource.
4. Integrate the method with your Lambda function.
5. Enable **Lambda proxy integration**.
6. Deploy the API to a stage named `dev`.

Invocation Example

```
curl https://<api-id>.execute-api.<region>.amazonaws.com/dev/rectangle
-H "Content-Type: application/json"
-d '{"length":4, "width":6}'
```

Expected Behavior

- API Gateway forwards the HTTP request to Lambda.
- Lambda processes the request and returns JSON output.
- API Gateway returns the response to the client.

Testing Requirements

Students must provide evidence of correct execution:

- Screenshot of Lambda test execution for Part 1.
- Screenshot of S3 bucket list (bucket names may be obscured).
- Screenshot of successful API Gateway invocation using `curl`.

Submission Requirements

Submit the following:

1. Python source code for all Lambda functions.
2. A single PDF containing:
 - Screenshots of Lambda tests.
 - Screenshots of API Gateway invocation.
 - The deployed API Gateway endpoint URL.

Grading Breakdown

- Part 1: Rectangle computation Lambda – 5 points
- Part 2: S3 bucket count Lambda – 5 points
- Part 3: API Gateway integration – 5 points

Guidance: Processing Input in a Lambda Function

This subsection provides guidance on how AWS Lambda processes input data when invoked through HTTP-based mechanisms such as a Function URL or API Gateway. The example shown here is for illustration only and is not related to the problems students are required to implement in this assignment.

The event Object

Every Lambda function is invoked with an **event** object, which is represented as a Python dictionary. When a Lambda function is triggered by an HTTP request, the request body is stored in:

`event["body"]`

Importantly, `event["body"]` is a **string**, even when the client sends JSON data.

Typical Input-Processing Pattern

A common pattern for handling JSON input in a Lambda function is:

1. Read the request body from `event["body"]`
2. Convert the JSON string into a Python dictionary
3. Extract required fields
4. Perform application logic
5. Return a JSON-formatted response

Illustrative Example

The following example accepts a user name and returns a greeting message.

Client Request

```
curl <function_url> \  
-H "Content-Type: application/json" \  
-d '{"name": "Alice"}'
```

Lambda Function Code

```
import json  
  
def lambda_handler(event, context):  
    body_str = event["body"]  
    body = json.loads(body_str)  
  
    name = body["name"]  
    message = f"Hello, {name}!"  
  
    return {  
        "statusCode": 200,  
        "body": json.dumps({  
            "message": message  
        })  
    }
```

Testing in the Lambda Console

When using the Lambda console *Test* feature, the input must reflect the structure used by HTTP invocations:

```
{  
  "body": "{\"name\": \"Alice\"}"  
}
```

The value associated with "body" must be a string, and quotation marks inside the JSON string must be escaped.

Handling Missing Input

To avoid runtime errors, input fields can be accessed defensively:

```
body = json.loads(event.get("body", "{}"))  
name = body.get("name", "Guest")
```

Key Takeaways

- The `event` object is always a Python dictionary.
- For HTTP-based invocations, user input is stored in `event["body"]`.
- `event["body"]` is a string and must be parsed using `json.loads()`.
- Responses should be returned as JSON strings using `json.dumps()`.